

Guia de Diseno v4.0 - Edicion Final

Valdris: El Nucleo Profundo

Documento operativo completo - Puntos 1, 2 y 3

Proyecto	Valdris: El Nucleo Profundo - Juego por Turnos JavaFX
Grupo	H12GEXTRA - Marcos Castro - Ventura Pacheco - Marino Rodriguez
Version	v4.0 Final - Sistema de turnos + IA enemiga + Mapa completo (33 salas)
Companion	Guia de Proyecto v3.0 (narrativa, arquitectura, advertencias tecnicas)

Contenido completo
* Punto 1: Sistema de turnos - fases, acciones, walkability, formula de combate, contadores. * Punto 2: IA enemiga - 8 subtipos con stats, pseudocodigo executeTurn() y notas. * Punto 3: Mapa de contenidos - 33 salas, grafos por zona, tamanos, enemigos y contenidos. * Puntos 4 y 5 (tabla de items y plan de 17 dias) se anaden en la siguiente sesion.

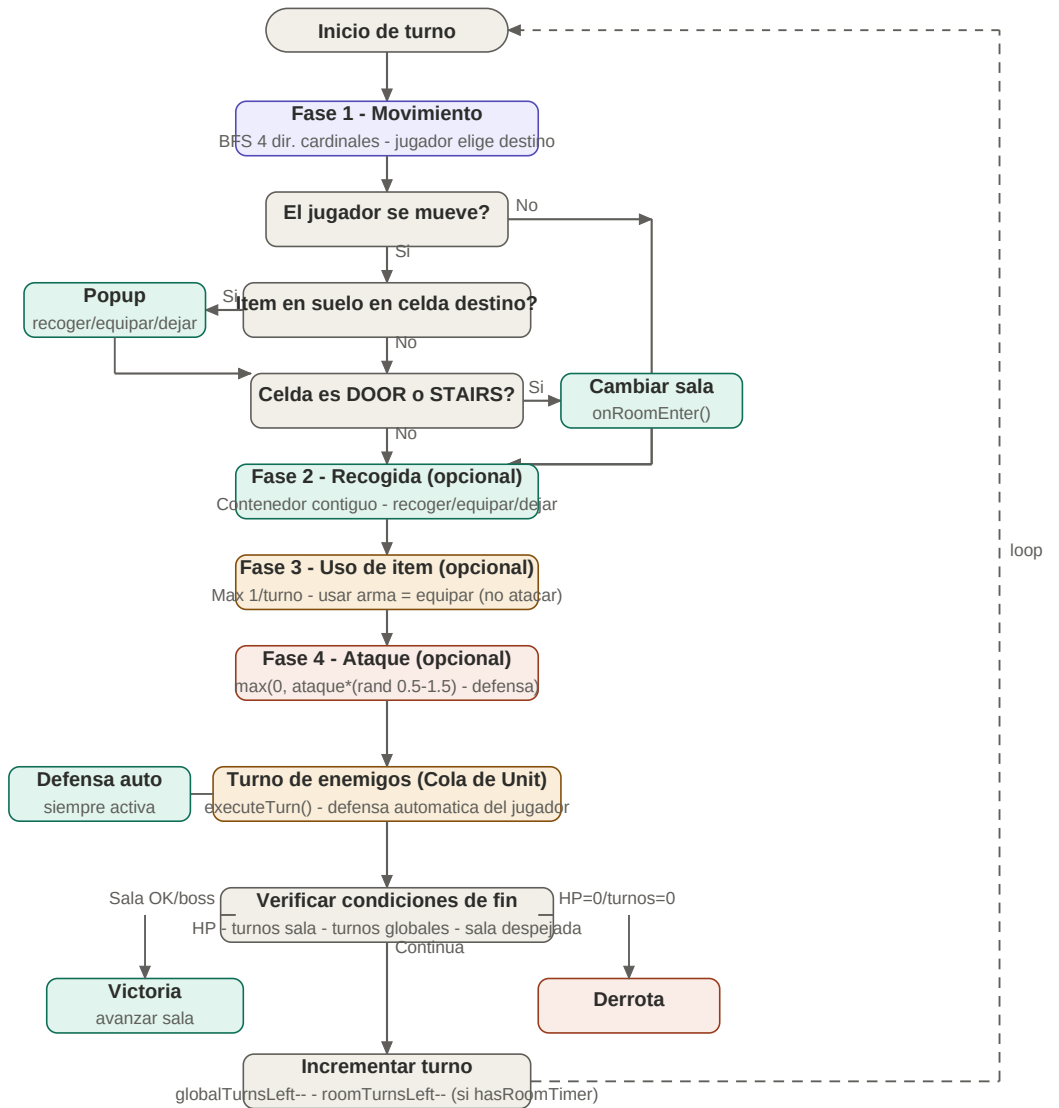
Punto 1 - Sistema de Turnos Completo

Referencia definitiva para implementar TurnManager.

1.0 Diagrama de flujo de un turno

Flujo completo con las 4 fases del jugador, turno de enemigos, verificación y loop.

FASE JUGADOR



FASE ENEMIGOS

FIN DE TURNO

Fase jugador
 Evento especial
 Turno enemigos
 Combate/derrota
 Control flujo

1.1 Acciones disponibles

Accion	Max/turno	Restriccion	Notas
Movimiento	1	Siempre primera	BFS 4 dir. Diagonal = 2 pasos BFS.
Recogida	1	Fase 1 (suelo) o Fase 2 (contenedor contiguo)	Item suelo: popup auto. Contenedor: accion manual Fase 2.
Uso de item	1	Solo inventario. Usar arma = equipar	No pocion Y ataque con arma mismo turno.
Ataque	1	Enemigo en rango BFS del arma	Sin arma: rango 1, danio base.
Ceder turno	--	En cualquier momento	Pasa al turno enemigo.

Caso especial - equipar vs atacar

- * Usar arma del inventario (Fase 3) = equiparla. NO es atacar.
- * Atacar con arma ya equipada (Fase 4) SI es la accion de ataque.
- * Se puede recoger una espada y atacar con ella en el mismo turno.

1.2 Fases del turno - orden fijo

Fase 1 - Movimiento

El jugador elige moverse o no. Siempre va primero.

- * BFSMovimiento.getCellsInRange() calcula celdas alcanzables.
- * JavaFX resalta celdas. El jugador hace clic en destino.
- * Si celda es DOOR/STAIRS: TurnManager.changeRoom().
- * Si celda tiene item en suelo: popup automatico recoger/equipar/dejar.
- * Si no se mueve: salta a Fase 2.

Fase 2 - Recogida (opcional)

Solo si hay contenedor (requiresAdjacent=true) en celda contigua.

- * TurnManager comprueba 4 celdas adyacentes (N,S,E,O).
 - * Si hay contenedor contiguo: ofrece recoger/equipar/guardar/ignorar.
 - * Si no hay contenedor: fase se salta.
- > Item en suelo = se pisa (Fase 1). Contenedor = celda adyacente, accion manual (Fase 2).

Fase 3 - Uso de ítem (opcional)

El jugador puede usar un ítem del inventario.

- * El jugador selecciona ítem del panel lateral JavaFX.
- * `Item.use(player)` ejecuta el efecto.
- * Usar arma = equiparla. No cuenta como ataque.
- * Solo 1 ítem por turno.

Fase 4 - Ataque (opcional)

El jugador puede atacar un enemigo en rango.

- * El jugador selecciona enemigo en rango.
- * $\text{Danio} = \max(0, \text{ataque} * (\text{rand}[0.5-1.5]) - \text{defensa})$.
- * Si muere: `Enemy.onDeath()` con `droplItem` si tiene.
- * Solo 1 ataque por turno.

1.3 Walkability y movimiento cardinal

Condicion	Transitable	Bloqueado
Tipo de celda	FLOOR, DOOR, DOOR_HIDDEN(activa), RUNE, LEVER, STAIRS	WALL, DOOR_LOCKED
Contenido	<code>unit==null</code> Y (<code>item==null</code> O <code>!requiresAdjacent</code>)	<code>unit!=null</code> O <code>item.requiresAdjacent==true</code>

```
// Cell.java
public boolean isWalkable() {
    if (type==WALL || type==DOOR_LOCKED) return false;
    if (unit != null) return false;
    if (item != null && item.isRequiresAdjacent()) return false;
    return true;
}
```

1.4 Formula de combate

Formula oficial de danio

- * $\text{vida_defensor} = \text{vida_defensor} - \max(0, \text{ataque} * \text{aleatorio} - \text{defensa})$
- * $\text{aleatorio} = \text{Math.random()} * 1.0 + 0.5 \rightarrow \text{rango } [0.5, 1.5]$
- * $\text{ataque} = \text{ataque base} + \text{modificadores arma} + \text{otros}$
- * $\text{defensa} = \text{defensa base} + \text{modificadores armadura} + \text{otros}$

```
public int calcularDanio(Unit atacante, Unit defensor) {
    double aleatorio = Math.random() * 1.0 + 0.5;
    int ataque = atacante.getAtaqueTotal();
```

```
int defensa = defensor.getDefensaTotal();
return Math.max(0, (int)(ataque * aleatorio) - defensa);
}
```

Caso	Aleatorio	Ejemplo (atq=20,def=5)	Resultado
Minimo	0.5	$\max(0, 20 \cdot 0.5 - 5) = 5$	5 de dano
Normal	1.0	$\max(0, 20 \cdot 1.0 - 5) = 15$	15 de dano
Critico	1.5	$\max(0, 20 \cdot 1.5 - 5) = 25$	25 de dano
Bloqueado	0.5	$\max(0, 20 \cdot 0.5 - 15) = 0$	0 de dano

1.5 Contadores y condiciones de fin

Contador	Donde	Cuando decrementa	Derrota si
Turnos globales	TurnManager.globalTurnsLeft	Siempre al fin de turno	==0
Turnos de sala	Room.roomTurnsLeft	Solo si Room.hasRoomTimer==true	==0

Condicion	Resultado	Quien detecta
HP jugador <= 0	Derrota	TurnManager tras takeDamage
globalTurnsLeft == 0	Derrota	TurnManager fin de turno
roomTurnsLeft == 0	Derrota	TurnManager fin de turno
Todos los enemigos muertos	Sala despejada	TurnManager tras onDeath
ParasitoEnemy muerto (Zona 5)	Victoria	TurnManager.triggerEnding

1.6 TurnManager - estructura

```
public class TurnManager {
    private Cola<Unit> turnQueue;
    private LSE<String> log;
    private GameState gameState;
    private int globalTurnsLeft;
    private TurnPhase currentPhase;
    public enum TurnPhase {
        PLAYER_MOVE, PLAYER_PICKUP, PLAYER_USE_ITEM, PLAYER_ATTACK,
        ENEMY_TURNS, CHECK_CONDITIONS, TURN_END
    }
    public void nextTurn() { ... }
    public void addLog(String msg) { ... }
    public void onRoomEnter() { ... }
    public void changeRoom(String dir) { ... }
    public void triggerEnding(CharacterType t) { ... }
    public void reconstruirReferencias() { ... }
}
```

1.7 Division de trabajo

Miembro	Bloque	Que incluye
Marino	Motor + integracion	Modelo completo (Fase 1), logica (Fase 2: BFS, TurnManager, combate, acertijos), integracion JavaFX.
Pacheco	Persistencia + tests	Fase 3 (LectorJSON, GameState, serializacion plana). Tests JUnit desde dia 1.
Castro	Interfaz JavaFX	Fase 4: GridPane, paneles, seleccion personaje, dialogos, finales. Empieza dia 10.

Regla de oro para Castro (JavaFX)

- * Los Controllers NO pueden contener logica de juego.
- * Un Controller solo llama a metodos de TurnManager o Player.
- * Si el metodo devuelve un int o modifica un modelo: no va en el Controller.

Punto 2 - IA Enemiga por Tipo y Subtipo

Define el comportamiento exacto de cada tipo y subtipo de enemigo.

Principios generales

- * Todas las IAs usan BFS real sobre Cell[][].
- * El BFS del jugador se cachea al inicio del turno de enemigos - 1 BFS por turno, no N.
- * BFSMovimiento.getCellInRange(row,col,range,room) se reutiliza para movimiento Y rango de ataque.
- * Movimiento cardinal (4 dir.). Diagonal = 2 pasos BFS naturalmente.
- * Linea de vision (Bresenham) requerida para Archer, Francotirador, Destructor y Controlador.

2.1 Linea de vision - Bresenham

```
public boolean tieneLineaDeVision(int r1,int c1,int r2,int c2,Room room){
    int dr=Math.abs(r2-r1), dc=Math.abs(c2-c1);
    int sr=(r1 < r2)?1:-1, sc=(c1 < c2)?1:-1;
    int err=dr-dc; int r=r1,c=c1;
    while(r!=r2||c!=c2){
        if(r!=r1||c!=c1){
            if(room.getCell(r,c).getType()==CellType.WALL) return false;
        }
        int e2=2*err;
        if(e2 > -dc){err-=dc;r+=sr;}
        if(e2 < dr){err+=dr;c+=sc;}
    }
    return true;
}
```

2.2 Sistema de efectos de estado

```
public class Efecto implements Comparable<Efecto> {
    public enum TipoEfecto { SLOW, BLIND, CURSE }
    private TipoEfecto tipo; private int turnosRestantes;
}

// TurnManager - inicio de cada turno del jugador:
public void decrementarEfectos(Player p) {
    LSE<Efecto> ef = p.getEfectosActivos();
    for(int i=0;i < ef.getSize();i++){
        ef.get(i).decrementarTurno();
        if(ef.get(i).getTurnosRestantes()==0) ef.del(ef.get(i));
    }
}
```


Efecto	Duracion	Impacto	Implementacion
SLOW	2 turnos	movePoints = Math.max(1, Math.ceil(original/2.0))	setMovePointsModificado(). Al expirar: restaurar.
BLIND	2 turnos	movePoints = Math.ceil(original/2.0) - Kael(3)->2, Syra(5)->3, Dorath(2)->1	Mismo mecanismo. JavaFX no muestra resaltado BFS.
CURSE	2 turnos	Danio recibido x1.5	En calcularDanio(): if tieneEfecto(CURSE) danio*=1.5

2.3 Familia Warrior

Warrior

Subtipo: Base - Warrior

Persigue al jugador por BFS y ataca si queda adyacente.

Stat	Valor	Stat	Valor
movePoints	2	attackRange	1
damage	15	defense	8
HP base	40	linea vision	No

```

executeTurn(player, room):
    camino = BFS(miPos, playerPos, room)
    mover min(movePoints, camino.size()-1) pasos
    si distBFS(miPos, playerPos) == 1: atacar(player)

```

Berserker

Subtipo: Subtipo - Warrior

Mas rapido y agresivo. Muy fragil. Invocado por el Mage Invocador.

Stat	Valor	Stat	Valor
movePoints	4	attackRange	1
damage	13	defense	3
HP base	25	linea vision	No

```

executeTurn(player, room):
    camino = BFS(miPos, playerPos, room)
    mover min(movePoints, camino.size()-1) pasos directo
    si distBFS(miPos, playerPos) == 1: atacar(player)

```

-> Invocado por el Mage Invocador con stats reducidos.

Guardian

Subtipo: Subtipo - Warrior

Defiende zona fija. Si jugador a BFS<=3 se acerca; si se aleja, vuelve a guardPosition.

Stat	Valor	Stat	Valor
movePoints	1	attackRange	1
damage	15	defense	18
HP base	60	linea vision	No

```
executeTurn(player, room):
    dist = distBFS(miPos, playerPos, room)
    si dist <= 3:
        mover 1 paso; si dist==1: atacar(player)
    sino si miPos != guardPosition:
        mover 1 paso hacia guardPosition
```

-> guardPosition asignado en spawn. Nunca mas de 3 celdas BFS de ella.

2.4 Familia Archer

Archer

Subtipo: Base - Archer

Zona de confort entre distancia minima y maxima. Huye si el jugador se acerca. Linea de vision obligatoria.

Stat	Valor	Stat	Valor
movePoints	3	attackRange	4
minComfortRange	2	damage	10
defense	4	HP base	30
linea vision	Si		

```
executeTurn(player, room):
    dist = distBFS(miPos, playerPos, room)
    si dist < minComfortRange:
        mover en direccion opuesta al jugador
    sino si dist <= attackRange Y lineaVision(...):
        atacar(player)
    sino:
        mover min(movePoints,pasos) acercandose
```

-> Direccion opuesta = invertir el primer paso del BFS hacia el jugador.

Francotirador

Subtipo: Subtipo - Archer

Mayor rango y dano. Dispara cada 2 turnos. El turno de cooldown lo usa para posicionarse.

Stat	Valor	Stat	Valor
movePoints	2	attackRange	5
minComfortRange	4	damage	15
defense	3	HP base	28
cooldown	2 turnos	linea vision	Si

```
executeTurn(player, room):  
  si cooldownRestante > 0: cooldownRestante--; posicionarse()  
  sino:  
    si dist <= attackRange Y lineaVision(...):  
      atacar(player); cooldownRestante = 2  
    sino: posicionarse()
```

-> *posicionarse() = misma logica que Archer base con minComfortRange=4.*

2.5 Familia Mage

Destructor

Subtipo: Subtipo Mage A - Area

No se mueve. Lanza AOE centrado en el jugador radio 2. No discrimina entre aliados y enemigos.

Stat	Valor	Stat	Valor
movePoints	0	attackRange	5
aoeRadius	2	damage	8/celda
defense	5	HP base	45
linea vision	Si - centro AOE		

```
executeTurn(player, room):  
  si distBFS(miPos,playerPos) <= attackRange  
  Y lineaVision(miPos,playerPos):  
    para cada celda en room:  
      si distManhattan(celda,playerPos) <= aoeRadius:  
        si celda.getUnit()!=null:  
          celda.getUnit().takeDamage(calcularDano(this,celda.getUnit()))
```

-> *distManhattan para el radio AOE. Linea de vision solo para el centro.*

Controlador

Subtipo: Subtipo Mage B - Efectos

Bajo dano. Aplica efecto aleatorio (SLOW/BLIND/CURSE). Si jugador ya tiene el efecto: reinicia duracion.

Stat	Valor	Stat	Valor
movePoints	2	attackRange	3
damage	4	defense	4
HP base	35	efectoDuracion	2 turnos
linea vision	Si		
<pre>executeTurn(player, room): si dist <= attackRange Y lineaVision(...): player.takeDamage(calcularDano(this,player)) efecto = elegirEfectoAleatorio() player.addEfecto(new Efecto(efecto,2)) sino: mover hacia player</pre>			

Invocador

Subtipo: Subtipo Mage C - Invocaciones

No ataca. Invoca Berserkers cada 2 turnos en celdas vacias adyacentes.

Stat	Valor	Stat	Valor
movePoints	2	attackRange	0
damage	0	defense	6
HP base	50	cooldown	2 turnos
linea vision	No		
<pre>executeTurn(player, room): cooldownRestante-- si cooldownRestante <= 0: celdas = getCeldasVaciasAdyacentes(miPos, radio=2) si celdas no vacia: mejor = celdaMasCercanaA(celdas, playerPos) berserker = new Berserker(mejor.row,mejor.col,room.getId()) room.addEnemy(berserker); turnQueue.addEnd(berserker) cooldownRestante = 2 mover min(movePoints,pasos) hacia player</pre>			
-> Solo invoca en celdas isWalkable()==true. Berserker actua desde el siguiente turno.			

2.6 Tabla resumen

Enemigo	Tipo	Mov e	Rng	Dani o	Def	HP	Especial
Warrior	Base Warrior	2	1	15	8	40	BFS hacia jugador
Berserker	Sub Warrior	4	1	13	3	25	Carga directa - invocado

Enemigo	Tipo	Mov e	Rng	Dani o	Def	HP	Especial
Guardian	Sub Warrior	1	1	15	18	60	Zona fija radio 3
Archer	Base Archer	3	4	10	4	30	Zona confort - linea vision
Francotirador	Sub Archer	2	5	15	3	28	Cooldown 2 turnos
Destructor	Sub Mage A	0	5	8x	5	45	AOE radio 2
Controlador	Sub Mage B	2	3	4	4	35	Efecto aleatorio 2 turnos
Invocador	Sub Mage C	2	0	0	6	50	Invoca Berserkers c/2 turnos

Punto 3 - Mapa de Contenidos por Zona

Las 33 salas del juego como Grafo bidireccional. Generacion aleatoria una vez al inicio, serializada a JSON. Al morir y recargar el mundo vuelve al estado inicial.

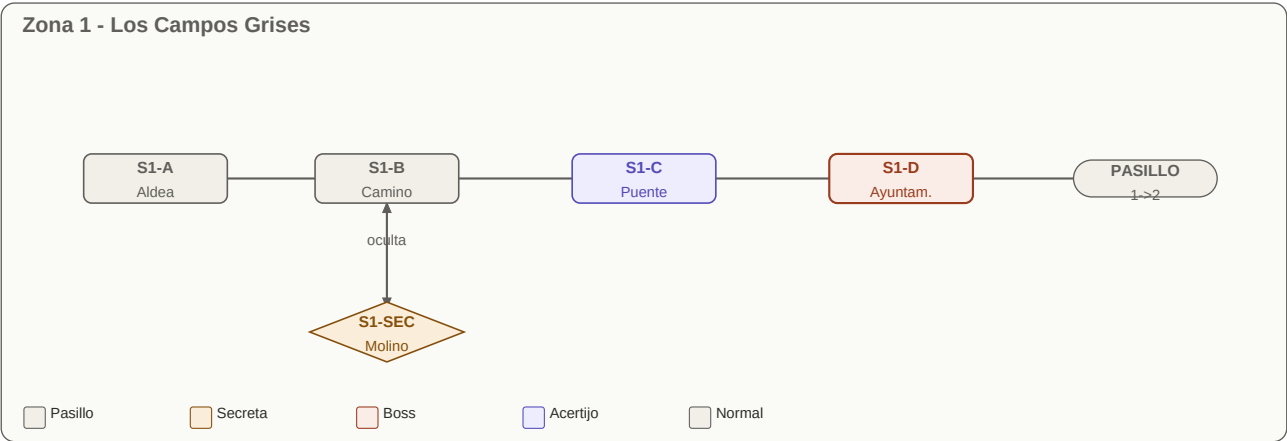
Principios del mapa

- * Grafo bidireccional. El jugador puede volver a cualquier sala visitada.
- * Excepcion unica: arista unidireccional Pasillo Final -> S5-D (punto de no retorno).
- * Enemigos fijos desde la generacion inicial. Estado persistente durante la partida.
- * Salas secretas: DOOR_HIDDEN, excepto S5-SEC que requiere Fragmento de Sello.
- * Pasillos de transicion: 3x8 fijo, hasRoomTimer=false, 1 item aleatorio zona siguiente.

3.1 Tipos de sala

Tipo	Rango filas	Rango cols	Cuando se usa
Pequena	5-6	5-6	Transicion, secretas, salas de item
Mediana	7-8	7-9	Combate estandar - la mayoria del juego
Grande	9-10	9-11	Acertijos, mini-boss, salas finales
Pasillo	3 (fijo)	8 (fijo)	Entre zonas. Sin enemigos, sin timer

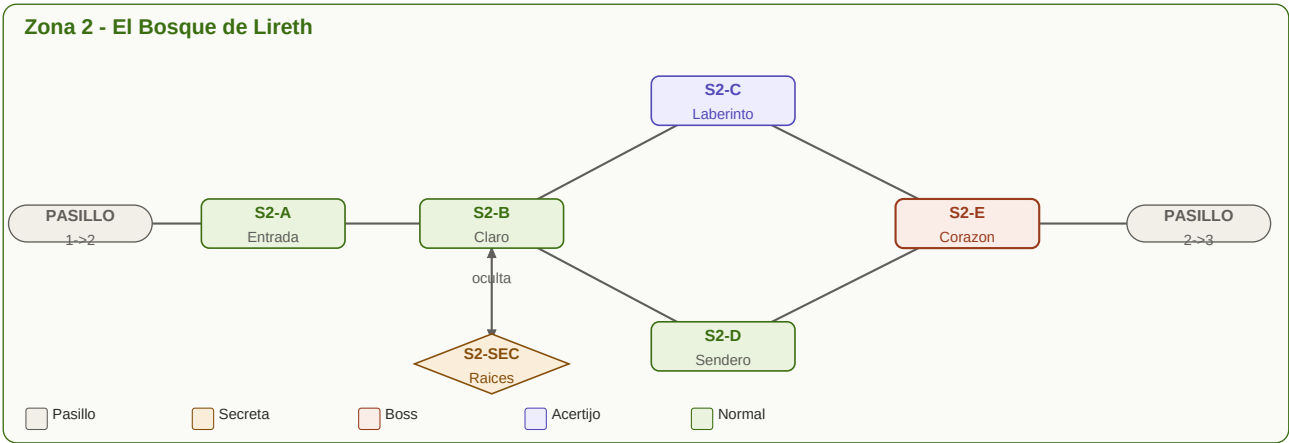
3.2 Zona 1 - Los Campos Grises



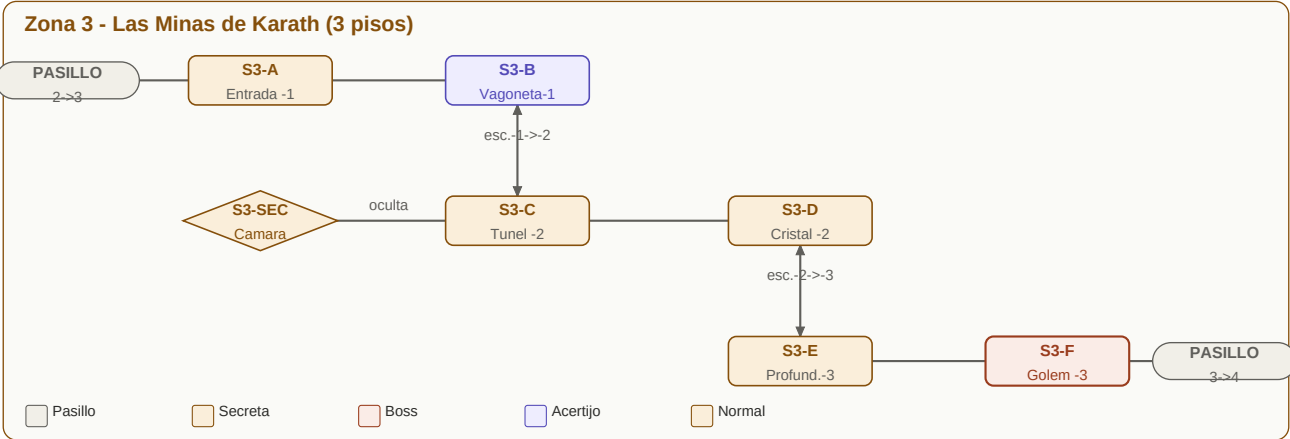
ID	Nombre	Tipo	Tamaño	Enemigos	Contenido especial
S1-A	Aldea Abandonada	Mediana	7x8	2 Warrior, 1 Archer	Tutorial implicito. Primera sala del juego
S1-B	El Camino Roto	Pequena	5x6	1 Warrior, 1 Archer	Bifurcacion hacia S1-SEC. Introduce movimiento tactico
S1-C	El Puente Gris	Grande	9x10	1 Controlador	Acertijo palancas del puente. Sin timer

ID	Nombre	Tipo	Tamaño	Enemigos	Contenido especial
S1-D	Ayuntamiento Corrupto	Grande	9x9	Mini-boss: Alcalde + 2 Warrior	Suelta Llave de Hierro al morir
S1-SEC	El Molino	Pequena	5x5	Ninguno	Cofre Poción de Fuerza. DOOR_HIDDEN desde S1-B
PASILLO 1->2	Linde del Bosque	Transicion	3x8	Ninguno	Item aleatorio Zona 2. hasRoomTimer=false

3.3 Zona 2 - El Bosque de Lireth

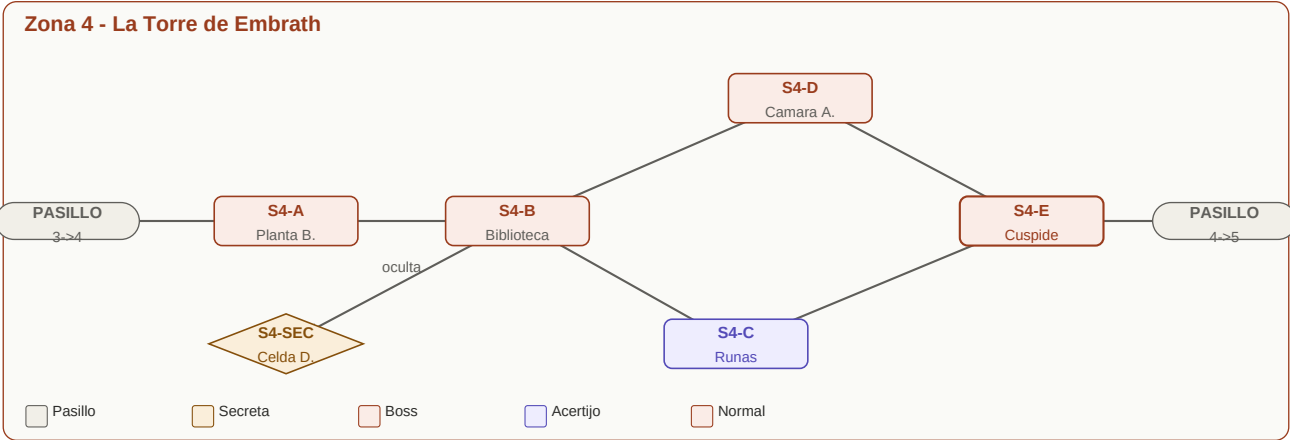


3.4 Zona 3 - Las Minas de Karath (3 pisos)



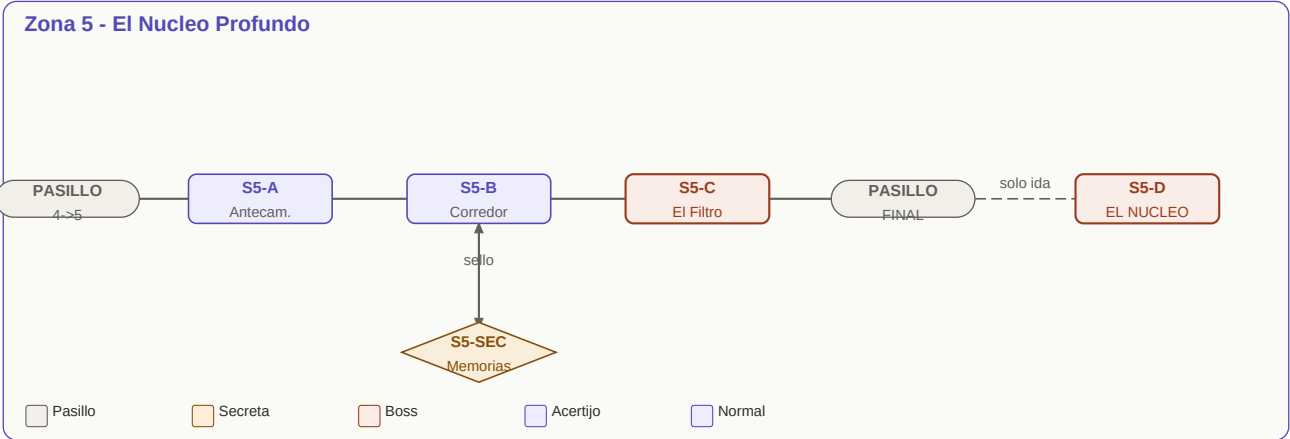
ID	Nombre	Piso	Taman o	Enemigos	Contenido especial
S3-A	Entrada de la Mina	-1	7x7	2 Warrior, 1 Berserker	Primera sala piso superior
S3-B	Sala de Vagonetas	-1	9x10	1 Invocador	Acertijo mecanismo vagoneta piso 1
S3-C	Tunel Central	-2	8x8	2 Guardian, 1 Archer	Escaleras a -1 y -3. Acceso a S3-SEC
S3-D	Camara de Cristal	-2	7x9	2 Destructor, 1 Berserker	Sala dificil - dos Destruyores
S3-E	Profundidades	-3	9x9	3 Warrior, 1 Francotirador	Primer Francotirador del juego
S3-F	Camara del Golem	-3	10x11	Mini-boss: Golem (2x2) + 2 Guardian	Suelta Fragmento de Sello
S3-SEC	Camara Enana	-2	5x6	Ninguno	Cofre Arma Zona 3. DOOR_HIDDEN desde S3-C
PASILLO 3->4	Base de la Torre	--	3x8	Ninguno	Item aleatorio Zona 4. hasRoomTimer=false

3.5 Zona 4 - La Torre de Embrath



ID	Nombre	Tipo	Tamaño	Enemigos	Contenido especial
S4-A	Planta Baja	Mediana	7x8	2 Constructo (Warrior), 1 Controlador	Kael tiene dialogo adicional aqui
S4-B	Biblioteca	Mediana	8x8	2 Francotirador, 1 Guardian	Acceso a S4-C, S4-D y S4-SEC
S4-C	Sala de Runas	Grande	9x10	1 Destructor, 1 Controlador	Acertijo suelo de runas. Sin timer
S4-D	Camara Alta	Pequena	6x6	3 Berserker	Sala rapida e intensa antes del boss
S4-E	Cuspide de la Torre	Grande	10x10	Mini-boss: Guardian Sin Nombre + 2 Constructo	Suelta Fragmento de Voluntad. Maestro de Kael
S4-SEC	La Celda de Dorath	Pequena	5x5	Ninguno	Dorath tiene dialogos adicionales. Suelta Pergamino Sellado. DOOR_HIDDEN desde S4-B
PASILLO 4->5	Descenso al Nucleo	Transicion	3x8	Ninguno	Item aleatorio Zona 5. hasRoomTimer=false

3.6 Zona 5 - El Nucleo Profundo



ID	Nombre	Tipo	Tamano	Enemigos	Contenido especial
S5-A	Antecamara	Mediana	7x7	2 Sombra Absorbida, 1 Eco de Magia	Primera sala del final
S5-B	Corredor de Sombras	Pequena	5x8	3 Sombra Absorbida, 1 Eco de Magia	Acceso a S5-SEC
S5-C	La Puerta del Filtro	Grande	9x9	El Filtro (Mage especial)	Ultimo combate antes de Malachar
S5-SEC	Camara de Memorias	Pequena	5x5	Ninguno	Lore de Malachar. Abre con Fragmento de Sello
PASILLO FINAL	El Umbral	Transicion	3x8	Ninguno	Aviso punto de no retorno. hasRoomTimer=false
S5-D	El Nucleo	Grande	10x11	Malachar (dialogo) + ParasitoEnemy	Sala final. Conversacion -> batalla -> desenlace

Pasillo Final - arista unidireccional

- * La arista Pasillo Final -> S5-D es la UNICA unidireccional del grafo.
- * Dungeon.connect(pasillo, S5-D, 'adelante') sin arista de retorno.
- * TurnManager muestra aviso explicito al jugador al entrar al Pasillo Final.
- * Desde S5-D: Dungeon.getAdjacentRooms() no devuelve el pasillo.

3.7 Resumen global

Zona	Normales	Secretas	Pasillos	Total	Predominante
Campos Grises	4	1	1	6	Mediana/Grande
Bosque de Lireth	5	1	1	7	Mediana/Grande
Minas de Karath	6	1	1	8	Mediana/Grande (3 pisos)
Torre de Embrath	5	1	1	7	Mediana/Grande
Nucleo Profundo	4	1	1	6	Pequena/Grande

Zona	Normales	Secretas	Pasillos	Total	Predominante
TOTAL	24	5	5	34	--

3.8 Generacion aleatoria y serializacion

Aleatorio al generar	Siempre fijo
Tamano exacto (dentro del rango del tipo)	Numero de salas y sus conexiones
Subtipo de enemigo por sala	Tipo base de enemigo por sala
Item concreto en pasillos de transicion	Cada pasillo tiene exactamente 1 item
Posicion de spawn de enemigos	Que enemigos aparecen en cada sala

```
// Nueva partida:
GameState estado = DungeonGenerator.generarMundo();
LectorJSON.guardar(estado, 'partida_nueva.json');

// Al morir y recargar:
GameState estado = LectorJSON.cargar('partida_nueva.json');
TurnManager.reconstruirReferencias(estado);
```